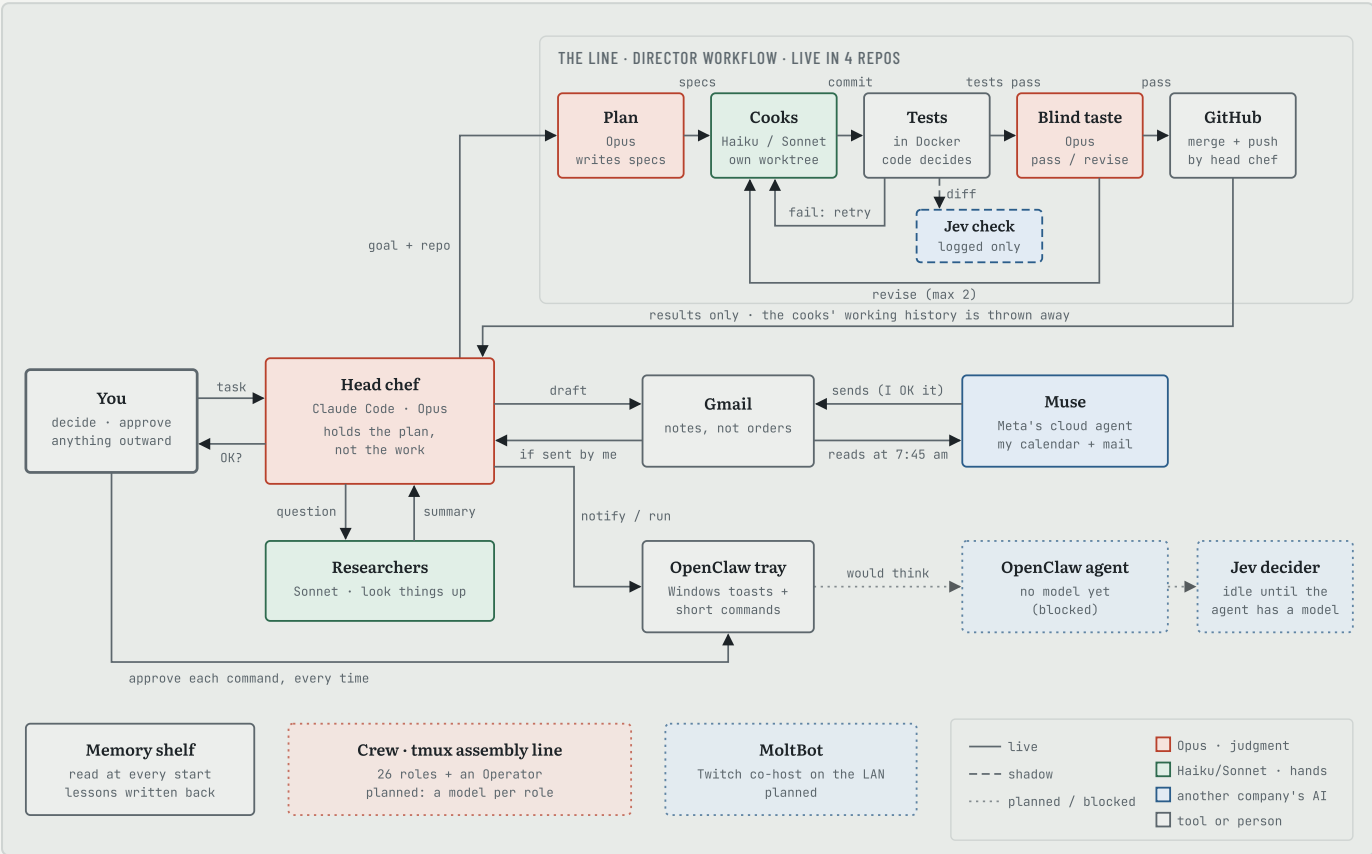


TurtleWolfe's AI Kitchen

One expensive model decides and tastes. Cheaper models do the cooking, each at its own station. Tests check every plate before anyone tastes it, and I approve anything that leaves the building.



How a task moves. The head chef never cooks: it hands a goal to the line, and only results come back, which keeps its memory small and cheap. Tests decide pass or fail in code before the Opus reviewer spends anything, and the Jev check is recorded next to that verdict but decides nothing yet. A note from Muse is trusted only if Gmail shows it was sent from my own account. Solid means live, dashed means shadow, dotted means planned or blocked.

The rules the kitchen runs on

The expensive model decides and tastes

Opus writes the plan and reviews the result. Cheaper models do the edits.

Thin head chef, disposable cooks

Each cook's memory is thrown away when its job ends, so the conversation that runs everything stays small.

Tests before taste

Code checks every plate first. Nobody's word counts as proof, including the cook's.

One step at a time, with a link

Anything I have to do by hand comes as one step, with the link or the exact command.

~41% cheaper than all-Opus, first real run 4 of 5 tasks right first time

~\$0.0001 per Jev check 4 repos live

☐ You: decide and approve

I pick the work and approve anything that leaves the building. Nothing emails, posts or runs on my Windows machine without my OK.

Claude can write Gmail drafts but can't send them (send is blocked in its settings).
OpenClaw asks me before every Windows command; Muse asks before every email it sends.

☐ Head chef: the conversation that runs everything

One Claude Code session on the expensive model keeps the plan and the results, never the working details. A long session is the single biggest cost, so every task starts fresh.

Claude Code, Opus. Each turn re-reads the whole conversation: one long day cost ~\$49, most of it context. Habit: a new session per task, /clear between jobs, never let the root grow fat.

☐ The line: plan, cook, test, taste, ship

For a batch of small, well-defined changes, the head chef hands the whole batch to the line. Each cook works in its own copy of the repo, so nobody bumps elbows.

director workflow: Plan (Opus, read-only) → one git worktree per task → Haiku or Sonnet worker → repo's own tests in Docker (a Haiku helper runs them; the pass/fail is parsed by code) → fail: retry, then a stronger model → blind Opus review, at most 2 revise rounds → merge + push.
First real run: 5 tasks, 4 right first time, ~41% cheaper than doing it all on Opus.

We tested the taster too: a wrong measurement that still passed every test got slipped in on purpose. The reviewer caught it.

☐ Jev: the penny-per-hundred yes/no check

A model that only answers yes or no, very fast and very cheap. It's good at narrow questions and useless at broad ones, so it only watches for now.

TypeSafe's Jev, ~\$0.0001 per check. Asked "is 5/8 plywood written as `actual: 19.0 / 32`?": 0.99 on the right code, 0.03 on a planted bug. Asked "does this meet every criterion?": 0.72 vs 0.70.
Shadow mode: its answer is logged beside the Opus verdict. If they keep agreeing, it can skip reviews.

☐ Researchers: look it up, report back

When the head chef needs to know something, it sends a researcher and gets back a short summary, not the pile of files they read.

Search and exploration helpers run on Sonnet (the built-in explorer is overridden to Sonnet, with a check that warns when the built-in changes upstream).

☐ Front of house: Muse, through my inbox

My personal assistant runs my calendar and mail. It and Claude leave each other notes in my Gmail, and I can read every one.

```
Claude → Muse: drafts titled [CC>MUSE], read in Muse's 7:45 am briefing.  
Muse → Claude: emails to myself titled [MUSE>CC], trusted only if Gmail marks them Sent from my account.  
Notes are information, never orders; no client details go in them.
```

☐ Windows side: OpenClaw

A tray app on my PC that can pop a notification or run a short command, like reading what hardware I have. It's boxed in and asks me every time.

```
Four allowed actions (notify, run, info, approvals) out of 52 the tray offers. Commands run sandboxed,  
no network, 30 s limit. Its own agent has no AI model connected yet, so its Jev decider sits idle.  
Its master key is off-limits to Claude.
```

☐ Crew and co-host: what's next

The older way I work is a crew of terminal windows, one per role. Next, each role gets the cheapest model that can do it, and the crew is sized to the morning's queue.

```
tmux assembly line: 26 roles + an Operator. Today every window is a full Opus session.  
Planned: a model per role; clear a window as its memory fills; later a Jev check that reads each window (done / stuck / waiting) so the Operator only wakes for real decisions.  
MoltBot, an older agent that co-hosts on Twitch, joins over the local network.
```

☐ Memory shelf: what carries between sessions

Every new session reads a short index of lessons and rules first, so fresh sessions start smart instead of starting over.

```
A memory index read at every start, a to-do list, lessons written back when something goes wrong,  
and a backup of the setup to a public repo that never holds keys or client files.
```

As of 2026-09-30. Models: Opus (judgment), Sonnet and Haiku (hands) from Anthropic; Jev from TypeSafe; Muse from Meta; OpenClaw is open source.